

14.5 节练习

练习 14.26: 为你的 StrBlob 类、StrBlobPtr 类、StrVec 类和 String 类定义下标运算符。

14.6 递增和递减运算符

在迭代器类中通常会实现递增运算符(++)和递减运算符(--),这两种运算符使得类可以在元素的序列中前后移动。C++语言并不要求递增和递减运算符必须是类的成员,但是因为它们改变的正好是所操作对象的状态,所以建议将其设定为成员函数。

对于内置类型来说,递增和递减运算符既有前置版本也有后置版本。同样,我们也应该为类定义两个版本的递增和递减运算符。接下来我们首先介绍前置版本,然后实现后置版本。

Best Practices

定义递增和递减运算符的类应该同时定义前置版本和后置版本。这些运算符通常应该被定义成类的成员。

定义前置递增/递减运算符

为了说明递增和递减运算符,我们不妨在 StrBlobPtr 类(参见 12.1.6 节,第 421 页)中定义它们:

```
class StrBlobPtr {
public:
    // 递增和递减运算符
    StrBlobPtr& operator++();           // 前置运算符
    StrBlobPtr& operator--();
    // 其他成员和之前的版本一致
};
```

Best Practices

为了与内置版本保持一致,前置运算符应该返回递增或递减后对象的引用。

567

递增和递减运算符的工作机理非常相似:它们首先调用 check 函数检验 StrBlobPtr 是否有效,如果是,接着检查给定的索引值是否有效。如果 check 函数没有抛出异常,则运算符返回对象的引用。

在递增运算符的例子中,我们把 curr 的当前值传递给 check 函数。如果这个值小于 vector 的大小,则 check 正常返回;否则,如果 curr 已经到达了 vector 的末尾,check 将抛出异常:

```
// 前置版本:返回递增/递减对象的引用
StrBlobPtr& StrBlobPtr::operator++()
{
    // 如果 curr 已经指向了容器的尾后位置,则无法递增它
    check(curr, "increment past end of StrBlobPtr");
    ++curr;           // 将 curr 在当前状态下向前移动一个元素
    return *this;
}
```

```
StrBlobPtr& StrBlobPtr::operator--()
{
    // 如果 curr 是 0, 则继续递减它将产生一个无效下标
    --curr;                      // 将 curr 在当前状态下向后移动一个元素
    check(curr, "decrement past begin of StrBlobPtr");
    return *this;
}
```

递减运算符先递减 `curr`, 然后调用 `check` 函数。此时, 如果 `curr` (一个无符号数) 已经是 0 了, 那么我们传递给 `check` 的值将是一个表示无效下标的非常大的正数值 (参见 2.1.2 节, 第 33 页)。

区分前置和后置运算符

要想同时定义前置和后置运算符, 必须首先解决一个问题, 即普通的重载形式无法区分这两种情况。前置和后置版本使用的是同一个符号, 意味着其重载版本所用的名字将是相同的, 并且运算对象的数量和类型也相同。

为了解决这个问题, 后置版本接受一个额外的 (不被使用) `int` 类型的形参。当我们使用后置运算符时, 编译器为这个形参提供一个值为 0 的实参。尽管从语法上来说后置函数可以使用这个额外的形参, 但是在实际过程中通常不会这么做。这个形参的唯一作用就是区分前置版本和后置版本的函数, 而不是真的要在实现后置版本时参与运算。

接下来我们为 `StrBlobPtr` 添加后置运算符:

```
class StrBlobPtr {
public:
    // 递增和递减运算符
    StrBlobPtr operator++(int);      // 后置运算符
    StrBlobPtr operator--(int);
    // 其他成员和之前的版本一致
};
```

Best
Practices

为了与内置版本保持一致, 后置运算符应该返回对象的原值 (递增或递减之前的值), 返回的形式是一个值而非引用。

568

对于后置版本来说, 在递增对象之前需要首先记录对象的状态:

```
// 后置版本: 递增/递减对象的值但是返回原值
StrBlobPtr StrBlobPtr::operator++(int)
{
    // 此处无须检查有效性, 调用前置递增运算时才需要检查
    StrBlobPtr ret = *this; // 记录当前的值
    ++*this;                // 向前移动一个元素, 前置++需要检查递增的有效性
    return ret;             // 返回之前记录的状态
}

StrBlobPtr StrBlobPtr::operator--(int)
{
    // 此处无须检查有效性, 调用前置递减运算时才需要检查
    StrBlobPtr ret = *this; // 记录当前的值
    --*this;                // 向后移动一个元素, 前置--需要检查递减的有效性
    return ret;             // 返回之前记录的状态
}
```

由上可知，我们的后置运算符调用各自的前置版本来完成实际的工作。例如后置递增运算符执行

```
++*this
```

该表达式调用前置递增运算符，前置递增运算符首先检查递增操作是否安全，根据检查的结果抛出一个异常或者执行递增 `curr` 的操作。假定通过了检查，则后置函数返回事先存好的 `ret` 的副本。因此最终的效果是，对象本身向前移动了一个元素，而返回的结果仍然反映对象在未递增之前原始的值。



因为我们不会用到 `int` 形参，所以无须为其命名。

显式地调用后置运算符

如在第 491 页介绍的，可以显式地调用一个重载的运算符，其效果与在表达式中以运算符的形式使用它完全一样。如果我们想通过函数调用的方式调用后置版本，则必须为它的整型参数传递一个值：

```
StrBlobPtr p(a1);           // p 指向 a1 中的 vector
p.operator++(0);             // 调用后置版本的 operator++
p.operator++();               // 调用前置版本的 operator++
```

尽管传入的值通常会被运算符函数忽略，但却必不可少，因为编译器只有通过它才能知道应该使用后置版本。

569

14.6 节练习

练习 14.27: 为你的 `StrBlobPtr` 类添加递增和递减运算符。

练习 14.28: 为你的 `StrBlobPtr` 类添加加法和减法运算符，使其可以实现指针的算术运算（参见 3.5.3 节，第 106 页）。

练习 14.29: 为什么不定义 `const` 版本的递增和递减运算符？

14.7 成员访问运算符

在迭代器类及智能指针类（参见 12.1 节，第 400 页）中常常用到解引用运算符（`*`）和箭头运算符（`->`）。我们以如下形式向 `StrBlobPtr` 类添加这两种运算符：

```
class StrBlobPtr {
public:
    std::string& operator*() const
    { auto p = check(curr, "dereference past end");
      return (*p)[curr];           // (*p) 是对象所指的 vector
    }

    std::string* operator->() const
    { // 将实际工作委托给解引用运算符
      return & this->operator*();
    }

    // 其他成员与之前的版本一致
}
```